

CloudShield AI / DAIL

06 — State & Invariant Lifecycle Specification

Version 1.0 | Final State/Lifecycle Baseline | 1 September 2026

Purpose: Define the legal lifecycle and state-transition semantics for Trusted State, Candidate State, Protected Invariants, verification evidence, invalidation, retry, rejection, escalation, and promotion. This document makes the trust model executable: it specifies which transitions are legal, which are forbidden, what evidence is required, and how state changes interact with invariant validity.

Document Control

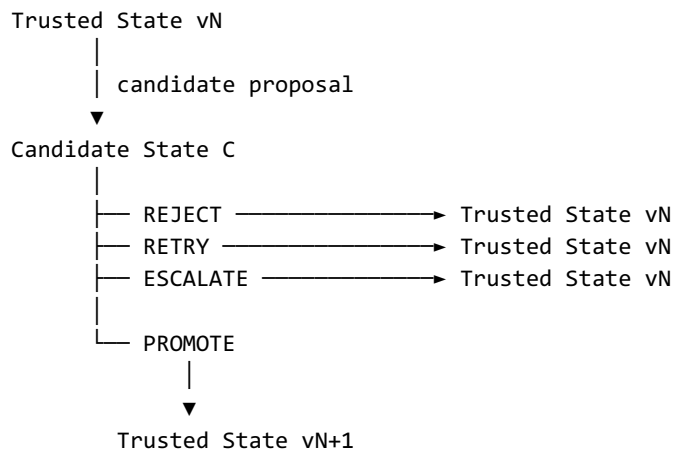
Field	Value
Document ID	CSAI-DAIL-LIFE-006
Status	FINAL STATE/LIFECYCLE BASELINE
Parent documents	01 SRS; 02 Architecture; 03 Functional Specification; 04 Terraform/AWS; 05 Domain/Data Model
Primary concern	State correctness and invariant evidence lifecycle
Trust boundary	Promotion Controller is the sole trusted-state advancement authority
Initial implementation	Deterministic fixed-patch DAIL

1. Purpose and Scope

DAIL is a state-transition system. A repair proposal is meaningful only relative to a known Trusted State and a known set of protected obligations. This specification therefore defines the lifecycle of the state and the validity of invariant evidence across transitions.

The central rule is: a candidate can be analyzed without becoming trusted, and an invariant can be protected only when current evidence supports it for the exact Trusted State it protects.

2. Core Lifecycle Model

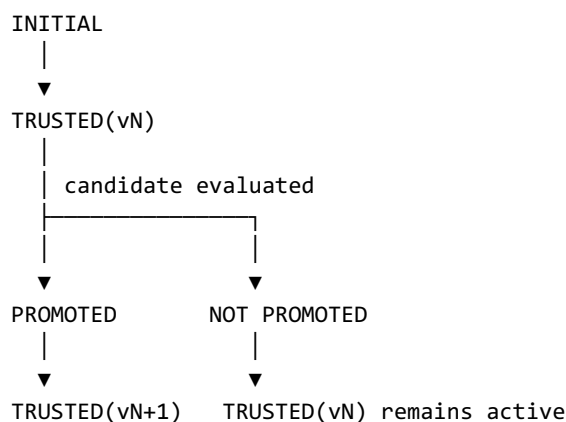


Promotion is the only transition that advances the trusted baseline. All non-promotion outcomes preserve the previous Trusted State.

3. State Taxonomy

State	Trust	Mutable?	Purpose
Trusted State	Trusted	No, except controlled version advancement	Authoritative baseline for future repair.
Candidate State	Untrusted	No after construction	Proposed transition under evaluation.
Historical Trusted State	Trusted historical record	Immutable	Audit/replay/research evidence.
Invariant Definition	Definition/policy	Versioned, not casually mutable	Defines a protected obligation.
Invariant Reference	State-specific	Updated through lifecycle	Connects an invariant definition to evidence for a Trusted State.

4. Trusted State Lifecycle



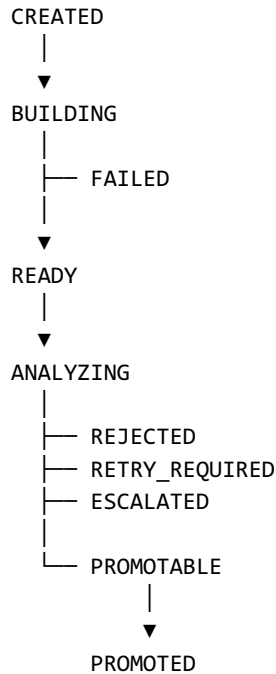
4.1 Creation Rules

- The initial trusted state must be explicitly established as a baseline fixture/state.
- A later Trusted State can be created only by successful promotion.
- Every promoted state has a parent Trusted State except the initial baseline.
- Each trusted state has a deterministic semantic hash.
- Each trusted state records the evidence supporting its protected invariants.

4.2 Immutability Rules

- Historical Trusted States cannot be edited to make a failed transition appear successful.
- Changing an invariant definition creates a new invariant version rather than rewriting historical meaning.
- Changing normalization semantics creates a new normalization version and requires explicit experimental handling.

5. Candidate State Lifecycle



5.1 Candidate Rules

- Candidate State always references one exact parent Trusted State version.
- Candidate State remains untrusted during all analysis phases.
- Candidate semantic content becomes immutable after construction.
- Candidate failure never mutates Trusted State.
- Promotion does not merely change a candidate flag; it creates a new Trusted State record with explicit lineage.

6. State Versioning

Property	Rule
Lineage	A logical chain of trusted states belongs to one lineage/context.
Version	Monotonically increases after successful promotion.
Parent	Every non-initial trusted state references its immediate parent.
Hash	Deterministic hash of canonical trusted semantic content.
Current pointer	Exactly one active current trusted version exists per lineage/context.
State candidate	A candidate based on an older parent cannot overwrite a newer current state.

7. Candidate-to-Trusted Transition Contract

```
PROMOTE(candidate C, parent vN)
requires:
  C.parent == current Trusted vN
  structural = PASS
  new requirement = PASS
```

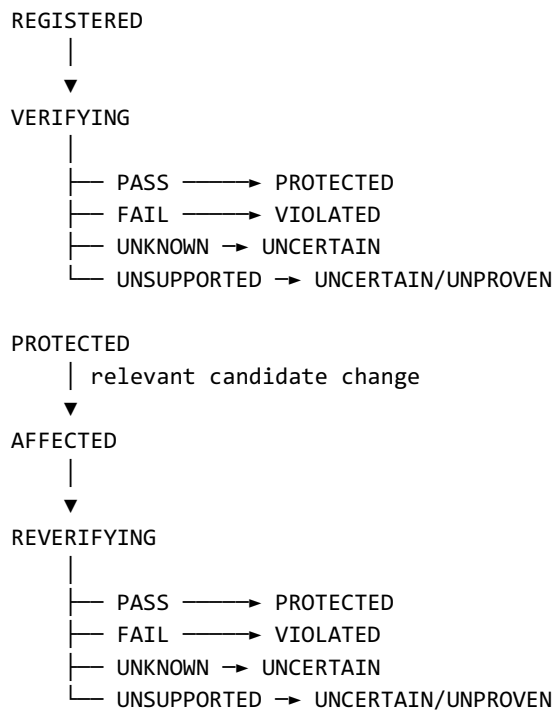
```

all required affected invariants = PASS
new-resource baseline checks = PASS
no blocking UNKNOWN/UNSUPPORTED/VERIFIER_ERROR
ensures:
  new Trusted vN+1 exists
  parent(vN+1) = vN
  C remains auditable as promoted candidate
  invariant evidence references vN+1

```

The exact decision matrix is owned by Document 09, but these lifecycle preconditions are mandatory.

8. Protected Invariant Lifecycle



The transition from PROTECTED to AFFECTED is caused by a relevant state transition, not by proof that the invariant is broken.

9. Invariant Status Semantics

Status	Meaning	Allowed next state
REGISTERED	Definition exists but has no current verification evidence for the target state.	VERIFYING
VERIFYING	Verification is currently being performed.	PROTECTED / VIOLATED / UNCERTAIN
PROTECTED	Current evidence establishes the invariant for the exact Trusted State.	AFFECTED after relevant change
AFFECTED	Prior evidence may be stale due to candidate impact.	REVERIFYING
REVERIFYING	Current candidate/trusted-state context is being checked.	PROTECTED / VIOLATED / UNCERTAIN

VIOLATED	Current evidence establishes that the invariant does not hold.	REVERIFYING after a new candidate is evaluated
UNCERTAIN	Current evidence is insufficient or verifier cannot establish the property.	REVERIFYING / explicit escalation

10. Evidence Validity Rule

Invariant evidence is state-specific. Evidence that proves INV-X for Trusted State vN does not automatically prove INV-X for Candidate State C or Trusted State vN+1.

```
Evidence validity:
valid(evidence, invariant, state) only if
  evidence.invariant_id/version matches
  AND evidence.target_state == state
  AND verifier/configuration context is compatible
  AND evidence is not invalidated by a relevant state transition
```

This is the core anti-stale-evidence rule.

11. Evidence Invalidation

11.1 Direct Invalidation

If a resource explicitly within an invariant's scope changes in a relevant way, the invariant becomes AFFECTED.

11.2 Dependency Invalidation

If a changed resource has a supported dependency path to an invariant's scoped resources/semantics, the invariant becomes AFFECTED.

11.3 Identity Uncertainty

If identity correspondence is uncertain for a resource relevant to a protected invariant, the existing evidence cannot be silently carried forward.

11.4 Dependency Uncertainty

If a relevant dependency is uncertain, the system must conservatively include the affected obligation in verification where required by policy.

12. Affected vs Violated

Concept	Meaning
AFFECTED	Evidence may be stale because the candidate changed relevant context. No claim about property truth is made.
VIOLATED	Current verification evidence establishes that the property is false/not satisfied.
PROTECTED	Current verification evidence establishes that the property holds for the exact trusted state.
UNCERTAIN	Current evidence cannot establish whether the property holds.

A DAIL implementation must never shortcut AFFECTED → VIOLATED or AFFECTED → PROTECTED without verification.

13. Invariant Scope Model

```
InvariantScope {  
  resources: [resource addresses / semantic selectors]  
  resource_types: [...]  
  relationships: [...]  
  properties: [...]  
  dependency_depth: defined/explicit  
}
```

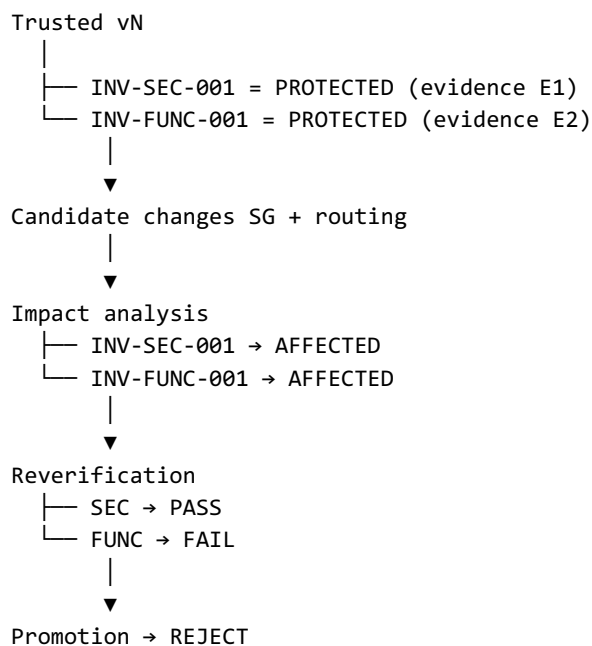
Scope determines what changes can invalidate evidence. Scope must be explicit enough that impact analysis can operate deterministically within the supported MVP.

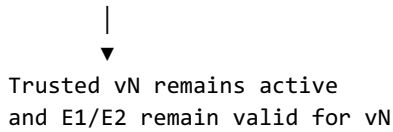
14. Protected Invariant Reference

```
InvariantRef {  
  invariant_id  
  invariant_version  
  target_state_id  
  status  
  evidence_ids  
  last_verified_at  
  invalidated_by_candidate_id | null  
  invalidation_reason | null  
}
```

- An invariant reference belongs to a specific Trusted State.
- The same invariant definition can have different statuses/evidence across state versions.
- Evidence references must point to evidence generated for the referenced state.
- An invalidated reference cannot be used to satisfy promotion for a later state.

15. State and Invariant Interaction





A rejected candidate does not consume or rewrite the evidence attached to the active Trusted State.

16. Promotion and Invariant Evidence

16.1 On PROMOTE

- Create Trusted State vN+1.
- Associate the candidate's normalized resources with vN+1.
- Carry forward unaffected protected invariant references only when their evidence remains valid for vN+1 under the defined policy.
- Associate newly verified affected invariants with current evidence for vN+1.
- Record the Promotion Decision and state-commit evidence.

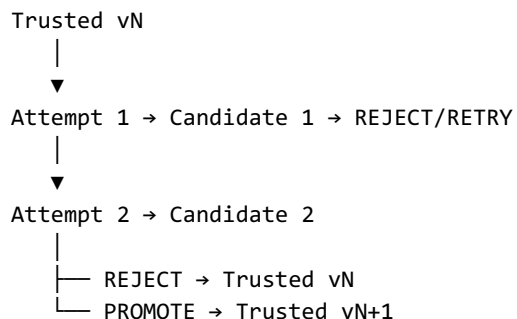
16.2 On REJECT

- Do not create Trusted State vN+1.
- Do not transfer candidate evidence into the active trusted state.
- Keep current invariant evidence associated with vN.
- Preserve candidate evidence for audit.

17. Carry-Forward Rules

Situation	Carry-forward behavior
Invariant unaffected and identity/dependency confidence is sufficient	May remain PROTECTED for new trusted state if policy explicitly permits evidence carry-forward.
Invariant directly affected	Must reverify before protection in new trusted state.
Invariant transitively affected	Must reverify before protection in new trusted state.
Identity uncertain	Do not silently carry evidence.
Dependency uncertain	Apply conservative invalidation/reverification policy.
Verifier configuration changed materially	Prior evidence may be incompatible; treat according to explicit evidence compatibility policy.
Invariant definition version changed	Old evidence cannot prove the new invariant version.

18. Retry Lifecycle



Every retry is anchored to the current trusted baseline. A rejected candidate is not the hidden starting point of the next core attempt.

19. Escalation Lifecycle

ESCALATE is a non-promoting terminal outcome for the automated attempt. The trusted baseline remains unchanged.

- Record the reason for escalation.
- Record unresolved uncertainty/errors.
- Preserve all candidate and verification evidence.
- Permit explicit human/experiment-controller handling outside the automatic promotion path.
- Do not mark the candidate trusted merely because escalation occurred.

20. Stale Parent Protection

A candidate may become stale if another evaluation promotes a different candidate first.

```
Before promotion:
    expected_parent = candidate.parent_state_id
    current_parent = trusted_state.current()

    if expected_parent != current_parent:
        reject promotion as STALE_PARENT
```

This rule prevents a candidate evaluated against v_N from overwriting a newer v_{N+1} .

21. Transactional Promotion

```
BEGIN
    verify parent is current
    validate promotion decision
    create TrustedState  $v_{N+1}$ 
    create/attach invariant refs and evidence refs
    commit decision
COMMIT

Failure → ROLLBACK
Trusted  $v_N$  remains current
```

Promotion is one logical state transition. Partial writes that imply a successful new trusted state are forbidden.

22. State Machine Rules

Rule	Required behavior
SM-001	TRUSTED state may advance only through PROMOTE.
SM-002	CANDIDATE cannot directly transition to TRUSTED without Promotion Controller authorization.
SM-003	REJECT preserves current Trusted State.
SM-004	RETRY preserves current Trusted State.

SM-005	ESCALATE preserves current Trusted State.
SM-006	AFFECTED invariant cannot be treated as PROTECTED for a new state without required verification.
SM-007	UNKNOWN cannot transition to PROTECTED when proof is required.
SM-008	UNSUPPORTED cannot transition to PROTECTED when proof is required.
SM-009	Historical state records are immutable.
SM-010	Stale-parent candidates cannot promote.

23. Invariant Evidence Compatibility

Evidence may be reused only when the evidence contract explicitly states that it remains valid across the transition. Compatibility must consider at minimum: target state, invariant definition version, verifier/version, relevant resource identity, relevant dependency context, and any runtime facts whose validity can expire.

Evidence condition	Compatibility
Exact same Trusted State	Valid if otherwise current.
New state with no relevant change	Potentially carryable if policy allows.
New state with relevant change	Not valid without re-verification.
New invariant version	Not valid for new version.
Identity uncertainty	Not safely reusable.
Runtime-only evidence with expired/changed runtime context	Not reusable without current verification.

24. Initial Baseline Establishment

Before iterative repair begins, the system must establish an initial Trusted State and its protected invariants.

1. Load the canonical baseline fixture.
2. Construct and normalize the baseline.
3. Verify the baseline security/functional conditions required by the experiment.
4. Create the initial Trusted State only when the baseline establishment protocol says it is acceptable.
5. Associate current invariant evidence with the initial state.
6. Persist baseline evidence before the first repair attempt.

The exact baseline acceptance protocol belongs to the experiment and verification specifications; this document defines the lifecycle requirement that a trusted baseline must have explicit evidence.

25. Canonical Regression Lifecycle

```
Baseline Trusted v0
├─ SEC = PROTECTED
└─ FUNC = PROTECTED
```

```
Candidate R1
├─ SEC = PASS
└─ FUNC = FAIL
```

↓

```
Promotion decision = REJECT
```

↓

```
Trusted v0 remains current
```

↓

Candidate R2 starts from Trusted v0

This scenario is a mandatory lifecycle test because it demonstrates that a locally successful security repair cannot corrupt the trusted baseline or erase a protected functional obligation.

26. Canonical Safe Promotion Lifecycle

Baseline Trusted v0

- └ SEC = PROTECTED
- └ FUNC = PROTECTED

Candidate S1

- └ SEC = PASS
- └ FUNC = PASS

↓

Promotion = PROMOTE

↓

Trusted v1

- └ SEC = PROTECTED (current evidence)
- └ FUNC = PROTECTED (current evidence)

27. Failure Scenarios

Scenario	Required lifecycle outcome
Malformed candidate	Candidate FAILED; Trusted State unchanged.
Security verifier FAIL	Candidate cannot promote.
Functional verifier FAIL	Candidate cannot promote if invariant is required.
Verifier UNKNOWN	Candidate cannot satisfy required proof.
Unsupported invariant	Candidate cannot satisfy required proof.
Identity uncertainty	Relevant evidence invalidation/reverification according to policy.
Dependency uncertainty	Conservative affected/reverification handling.
Database commit failure	No new current Trusted State.
Stale parent	Promotion rejected; current state unchanged.
LLM failure	No state advancement.

28. Lifecycle Events

Event	Minimum fields
STATE_CREATED	state_id, lineage, version, parent, hash, timestamp
CANDIDATE_CREATED	candidate_id, parent_state_id, patch_id, source, timestamp
INVARIANT_AFFECTED	invariant_id/version, state_id, candidate_id, reason
VERIFICATION_COMPLETED	verification_id, target, result, verifier/version
PROMOTION_DECIDED	decision_id, candidate_id, parent, decision, policy/version
STATE_PROMOTED	new_state_id, parent_state_id, decision_id, timestamp
PROMOTION_FAILED	candidate/parent, reason, timestamp
RETRY_REQUESTED	attempt, parent_state_id, reason
ESCALATED	candidate/attempt, reason, unresolved conditions

29. Lifecycle Audit Requirements

- Every state transition must be reconstructable from events and persisted records.
- Every invariant status transition must have a reason/evidence reference.
- Promotion decisions must reference the exact parent and candidate.
- Rejected candidate records must remain available.
- Evidence invalidation must identify the triggering candidate/change.
- Historical state/evidence records must not be silently rewritten.

30. Implementation Rules

- Use explicit state/status enums, not ad hoc booleans.
- Represent transitions through domain services/state-machine functions.
- Reject illegal transitions at the domain boundary.
- Use repository transactions for promotion.
- Make current-state lookup explicit and version-aware.
- Do not permit persistence-layer convenience methods to bypass lifecycle validation.
- Keep candidate analysis read-only with respect to Trusted State.

31. Lifecycle Test Matrix

Test	Expected result
Create candidate from vN	Candidate.parent = vN; Trusted unchanged.
Reject candidate	Current state remains vN.
Retry candidate	Next candidate parent remains vN.
Promote candidate	Create vN+1 with parent vN.
Promote stale candidate	Reject as stale; vN/current newer state unchanged.
Affect protected invariant	Status becomes AFFECTED.
Reverify affected invariant PASS	New state can reference PROTECTED evidence.
Reverify affected invariant FAIL	Invariant becomes VIOLATED; promotion blocked when required.
UNKNOWN reverify	Invariant remains UNCERTAIN; promotion blocked when proof required.
Unsupported verifier	Invariant remains unproven; promotion blocked when proof required.
Promotion DB failure	No partial trusted advancement.

32. Lifecycle Acceptance Criteria

- Trusted State advancement occurs only through PROMOTE.
- Candidate State never becomes trusted by mutation of a status field alone.
- Rejected candidates do not become trusted parents.
- Every Trusted State has explicit parent/version/hash/evidence lineage.
- Protected invariant evidence is state-specific.
- Relevant changes invalidate protected evidence.
- Identity/dependency uncertainty is conservative.
- UNKNOWN/UNSUPPORTED cannot satisfy required proof.
- Stale-parent promotion is impossible.
- Promotion is transactional.
- All lifecycle transitions are auditable.

33. Relationship to Other Specifications

Document	Lifecycle dependency
05 — Domain & Data Model	Defines state/invariant/evidence objects persisted by this lifecycle.
07 — Identity & Dependency	Determines correspondence/dependency facts that trigger invalidation.
08 — Impact & Invalidation	Computes affected invariant set used by lifecycle transitions.
09 — Verification & Promotion	Defines exact verification and promotion policy/matrix.
11 — Evidence & Logging	Defines event/evidence persistence and audit format.
12 — Testing & QA	Turns lifecycle acceptance criteria into automated tests.

34. Final State/Lifecycle Directive

The DAIL lifecycle is intentionally conservative. Trusted State is a historical, evidence-backed baseline; Candidate State is an isolated proposal; Protected Invariant status is a claim tied to a specific trusted state and current evidence. Any transition that would weaken these distinctions is architecturally invalid.

The implementation must therefore make illegal state transitions difficult to express, reject them at domain boundaries, and preserve complete evidence for both successful and unsuccessful candidate evaluations.

Appendix A — Legal Transition Summary

Trusted:

INITIAL → TRUSTED

TRUSTED(vN) --PROMOTE--> TRUSTED(vN+1)

Candidate:

CREATED → BUILDING → READY → ANALYZING

ANALYZING → PROMOTED | REJECTED | RETRY_REQUIRED | ESCALATED

Invariant:

REGISTERED → VERIFYING

VERIFYING → PROTECTED | VIOLATED | UNCERTAIN

PROTECTED → AFFECTED

AFFECTED → REVERIFYING

REVERIFYING → PROTECTED | VIOLATED | UNCERTAIN

Forbidden:

Candidate → Trusted without Promotion Controller

Rejected Candidate → Trusted Parent

UNKNOWN → PROTECTED (when proof required)

UNSUPPORTED → PROTECTED (when proof required)

Stale Candidate → Current Trusted State

Appendix B — Baseline Authority

This specification formalizes the state/invariant lifecycle implied by Documents 01–05. It preserves the established DAIL trust boundary, deterministic-first implementation sequence, protected invariant model, conservative uncertainty handling, state lineage, and evidence-driven promotion semantics.